

AI-ASSIGNMENT-12.4

NAME: NIPUN.I

H.T.NO: 2303A52208

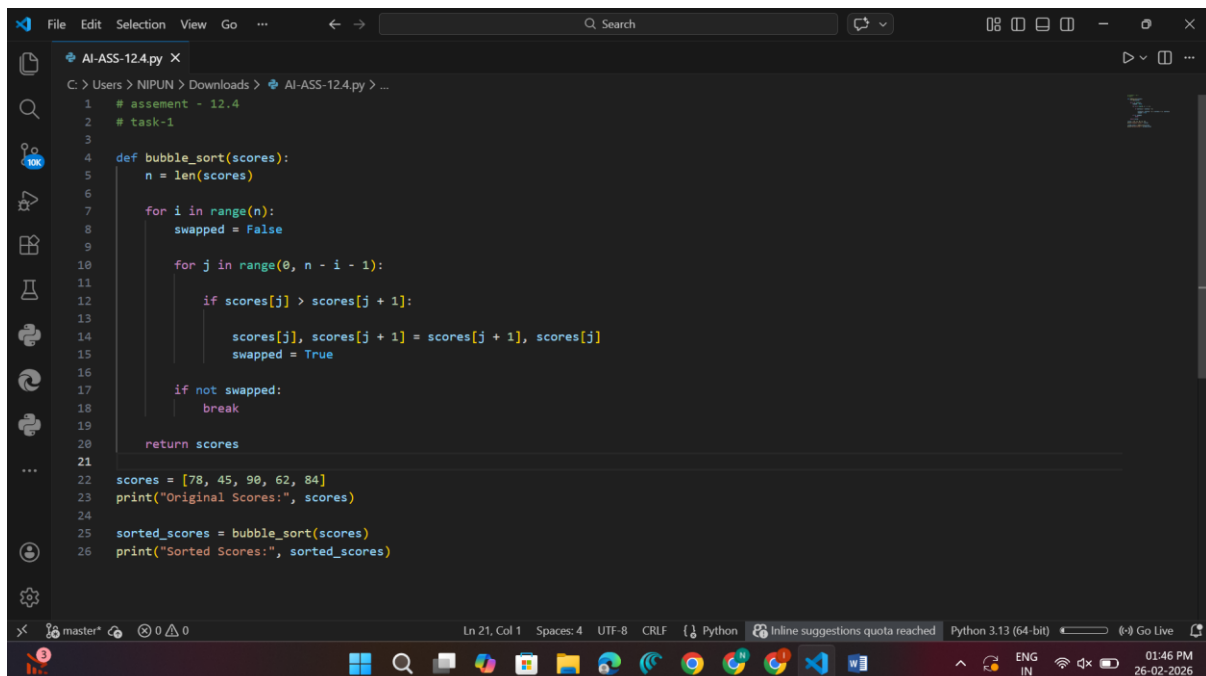
TASK-1:

Prompt:

Write a Python program to implement **Bubble Sort** for sorting student exam scores.
Use AI assistance to:

- Add inline comments explaining comparisons, swaps, and passes.
- Implement early termination if the list becomes sorted.
- Provide time complexity analysis.
- Show sample input and output.

Code:



```
1 # assement - 12.4
2 # task-1
3
4 def bubble_sort(scores):
5     n = len(scores)
6
7     for i in range(n):
8         swapped = False
9
10        for j in range(0, n - i - 1):
11
12            if scores[j] > scores[j + 1]:
13
14                scores[j], scores[j + 1] = scores[j + 1], scores[j]
15                swapped = True
16
17        if not swapped:
18            break
19
20    return scores
21
22 scores = [78, 45, 90, 62, 84]
23 print("Original Scores:", scores)
24
25 sorted_scores = bubble_sort(scores)
26 print("Sorted Scores:", sorted_scores)
```

Output:

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-12.4.py
Original Scores: [78, 45, 90, 62, 84]
Sorted Scores: [45, 62, 78, 84, 90]
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation:

Bubble Sort works by repeatedly comparing adjacent elements and swapping them if needed.

- **Best Case (Already Sorted)** → $O(n)$ (due to early termination)
- **Average Case** → $O(n^2)$
- **Worst Case (Reverse Sorted)** → $O(n^2)$

Bubble Sort is suitable only for small datasets. It is inefficient for large data but easy to understand and implement.

=====

TASK-2:

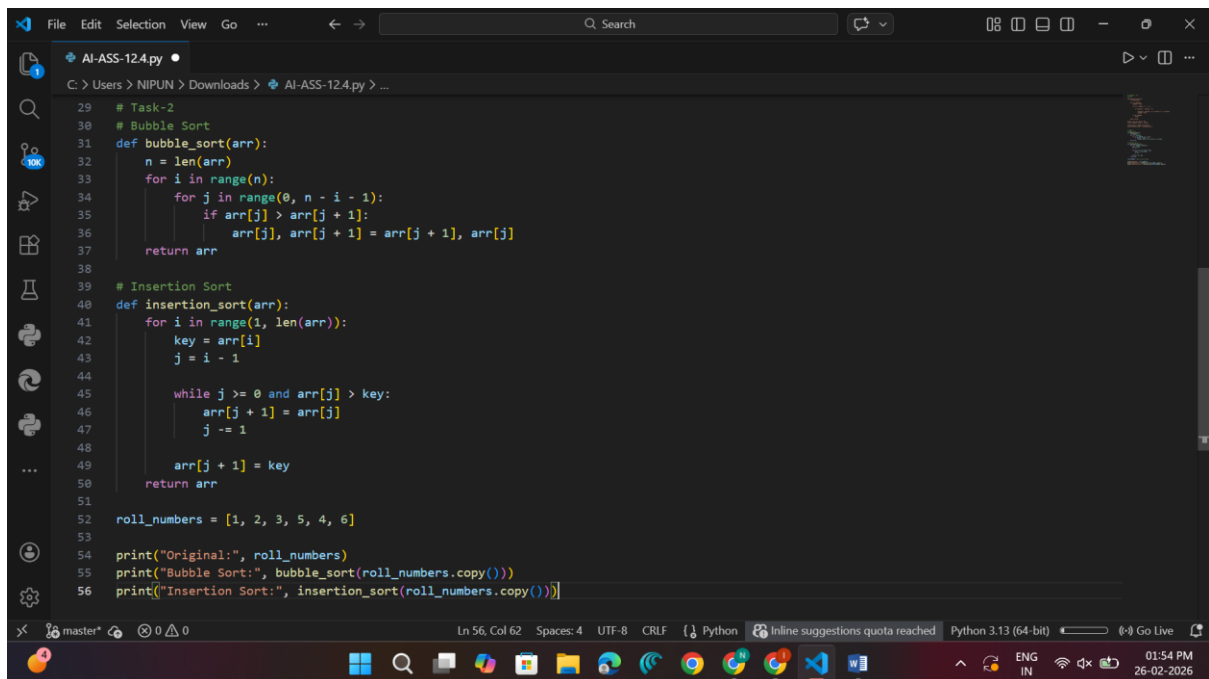
Prompt:

Given a nearly sorted list of roll numbers, start with Bubble Sort.

Ask AI to suggest a better algorithm.

Implement **Insertion Sort** and explain why it performs better on nearly sorted data.

Code:

A screenshot of a Python IDE window titled 'AI-ASS-124.py'. The code defines two sorting functions: 'bubble_sort' and 'insertion_sort'. The 'bubble_sort' function iterates through a list, comparing adjacent elements and swapping them if they are in the wrong order. The 'insertion_sort' function iterates through a list, inserting each element into its correct position in the sorted portion of the list. The code also includes a list of roll numbers [1, 2, 3, 5, 4, 6] and prints the original list, the result of bubble_sort, and the result of insertion_sort. The IDE interface shows a search bar, a file explorer on the left, and a status bar at the bottom indicating the file is on line 56, column 62, with 4 spaces, UTF-8 encoding, and CRLF line endings. The status bar also shows 'Python 3.13 (64-bit)' and 'Go Live' button.

```
29 # Task-2
30 # Bubble Sort
31 def bubble_sort(arr):
32     n = len(arr)
33     for i in range(n):
34         for j in range(0, n - i - 1):
35             if arr[j] > arr[j + 1]:
36                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
37     return arr
38
39 # Insertion Sort
40 def insertion_sort(arr):
41     for i in range(1, len(arr)):
42         key = arr[i]
43         j = i - 1
44
45         while j >= 0 and arr[j] > key:
46             arr[j + 1] = arr[j]
47             j -= 1
48
49         arr[j + 1] = key
50     return arr
51
52 roll_numbers = [1, 2, 3, 5, 4, 6]
53
54 print("Original:", roll_numbers)
55 print("Bubble Sort:", bubble_sort(roll_numbers.copy()))
56 print("Insertion Sort:", insertion_sort(roll_numbers.copy()))
```

Output:

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-12.4.py
Original: [1, 2, 3, 5, 4, 6]
Bubble Sort: [1, 2, 3, 4, 5, 6]
Insertion Sort: [1, 2, 3, 4, 5, 6]
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> █
```

Observation:

Insertion Sort performs better on nearly sorted data because:

- It only shifts misplaced elements.
- It does not repeatedly compare already sorted parts.
- Best case complexity is **$O(n)$** .

Algorithm	Nearly Sorted	Random Data
Bubble Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$

Insertion Sort is preferred when data is partially sorted.

=====

Task-3:

Prompt:

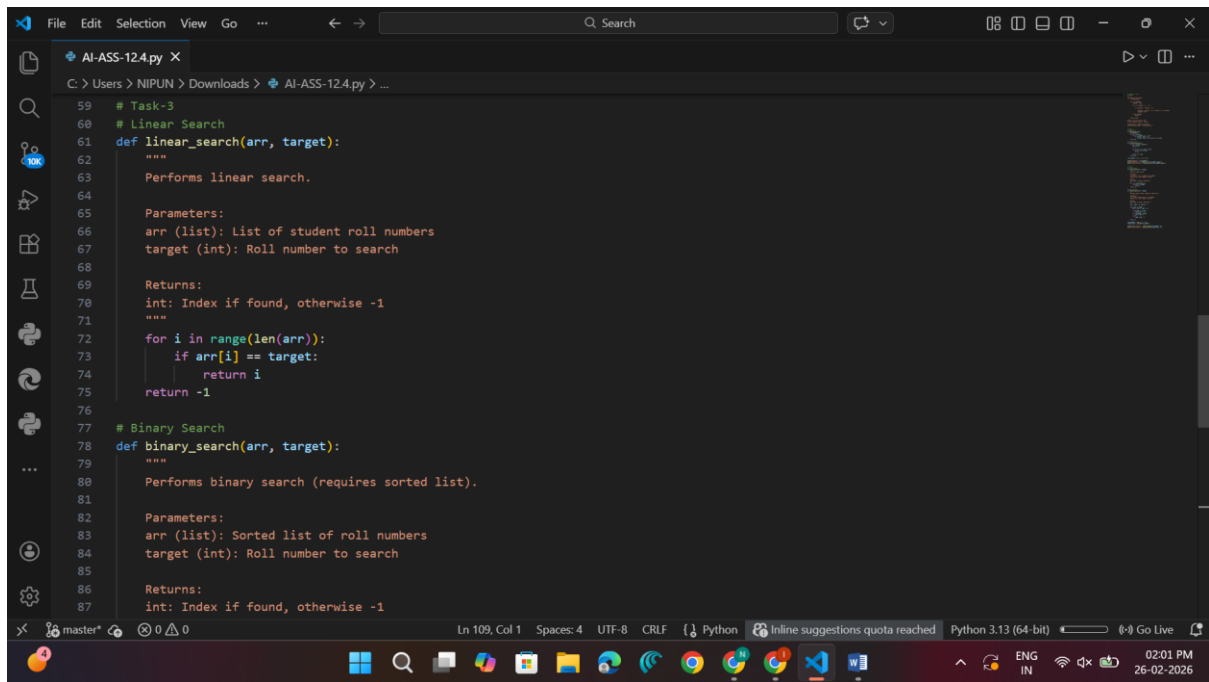
Implement:

- Linear Search (for unsorted data)
- Binary Search (for sorted data)

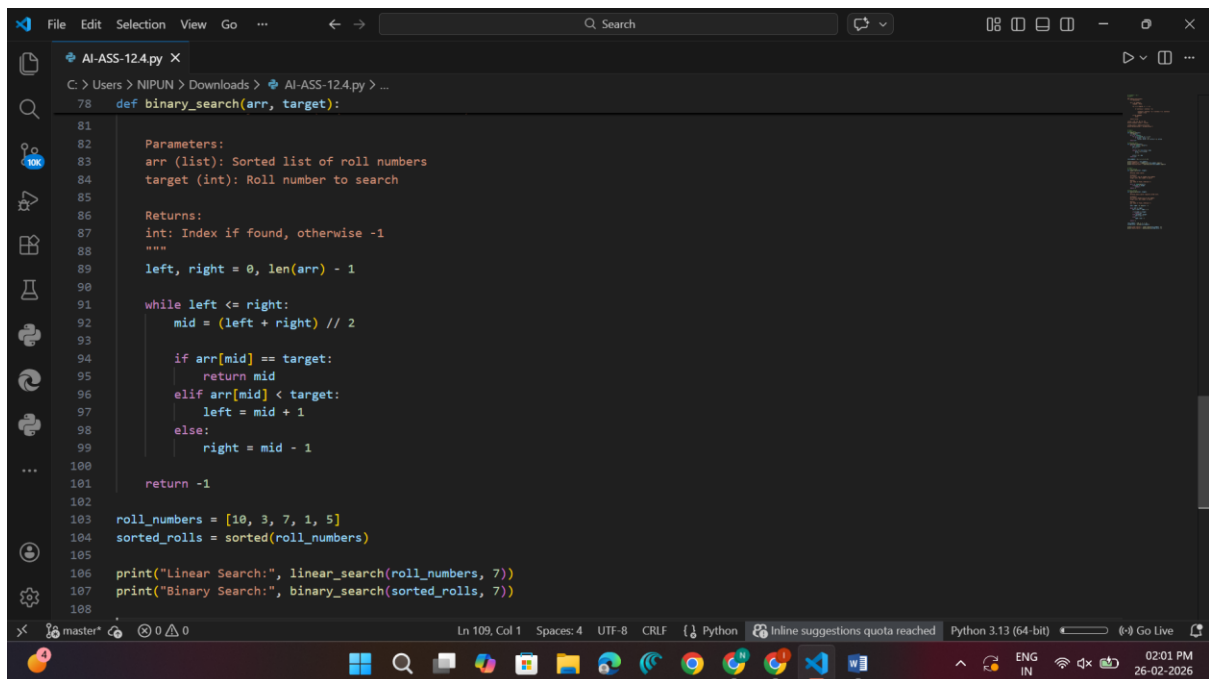
Add:

- Proper docstrings
- Complexity explanation
- Use-case comparison

Code:



```
59 # Task-3
60 # Linear Search
61 def linear_search(arr, target):
62     """
63     Performs linear search.
64
65     Parameters:
66     arr (list): List of student roll numbers
67     target (int): Roll number to search
68
69     Returns:
70     int: Index if found, otherwise -1
71     """
72     for i in range(len(arr)):
73         if arr[i] == target:
74             return i
75     return -1
76
77 # Binary Search
78 def binary_search(arr, target):
79     """
80     Performs binary search (requires sorted list).
81
82     Parameters:
83     arr (list): Sorted list of roll numbers
84     target (int): Roll number to search
85
86     Returns:
87     int: Index if found, otherwise -1
88     """
```



```
81
82     Parameters:
83     arr (list): Sorted list of roll numbers
84     target (int): Roll number to search
85
86     Returns:
87     int: Index if found, otherwise -1
88     """
89     left, right = 0, len(arr) - 1
90
91     while left <= right:
92         mid = (left + right) // 2
93
94         if arr[mid] == target:
95             return mid
96         elif arr[mid] < target:
97             left = mid + 1
98         else:
99             right = mid - 1
100
101     return -1
102
103 roll_numbers = [10, 3, 7, 1, 5]
104 sorted_rolls = sorted(roll_numbers)
105
106 print("Linear Search:", linear_search(roll_numbers, 7))
107 print("Binary Search:", binary_search(sorted_rolls, 7))
108
```

Output:

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-12.4.py
Linear Search: 2
Binary Search: 3
```

Observation

- Linear Search $\rightarrow O(n)$
- Binary Search $\rightarrow O(\log n)$

- Linear Search → Unsorted data
- Binary Search → Sorted data only

Binary Search is much faster for large sorted datasets.

=====

TASK-4

Prompt:

Complete recursive Quick Sort and Merge Sort functions.
Add docstrings and explain recursion.
Compare performance for random, sorted, and reverse data.

Code:

```

110 # Task-4
111 # Quick Sort
112 def quick_sort(arr):
113     if len(arr) <= 1:
114         return arr
115
116     pivot = arr[len(arr) // 2]
117
118     left = [x for x in arr if x < pivot]
119     middle = [x for x in arr if x == pivot]
120     right = [x for x in arr if x > pivot]
121
122     return quick_sort(left) + middle + quick_sort(right)
123
124
125 # Merge Sort
126 def merge_sort(arr):
127     if len(arr) <= 1:
128         return arr
129
130     mid = len(arr) // 2
131     left = merge_sort(arr[:mid])
132     right = merge_sort(arr[mid:])
133
134     return merge(left, right)
135
136
137 def merge(left, right):
138     result = []

```

```

137 def merge(left, right):
141     while i < len(left) and j < len(right):
142         if left[i] < right[j]:
143             result.append(left[i])
144             i += 1
145         else:
146             result.append(right[j])
147             j += 1
148
149     result.extend(left[i:])
150     result.extend(right[j:])
151
152     return result
153
154 random_data = [8, 3, 5, 1, 9, 6]
155 sorted_data = [1, 2, 3, 4, 5, 6]
156 reverse_data = [6, 5, 4, 3, 2, 1]
157
158 print("---- RANDOM DATA ----")
159 print("Quick Sort:", quick_sort(random_data))
160 print("Merge Sort:", merge_sort(random_data))
161
162 print("\n---- SORTED DATA ----")
163 print("Quick Sort:", quick_sort(sorted_data))
164 print("Merge Sort:", merge_sort(sorted_data))
165
166 print("\n---- REVERSE SORTED DATA ----")
167 print("Quick Sort:", quick_sort(reverse_data))
168 print("Merge Sort:", merge_sort(reverse_data))

```

Output:

```

PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-12.4.py
---- RANDOM DATA ----
Quick Sort: [1, 3, 5, 6, 8, 9]
Merge Sort: [1, 3, 5, 6, 8, 9]

---- SORTED DATA ----
Quick Sort: [1, 2, 3, 4, 5, 6]
Merge Sort: [1, 2, 3, 4, 5, 6]

---- REVERSE SORTED DATA ----
Quick Sort: [1, 2, 3, 4, 5, 6]
Merge Sort: [1, 2, 3, 4, 5, 6]
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>

```

Observation

Algorithm	Best	Average	Worst
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

= > Quick Sort → Faster in practice, good for random data

= > Merge Sort → Stable and consistent performance

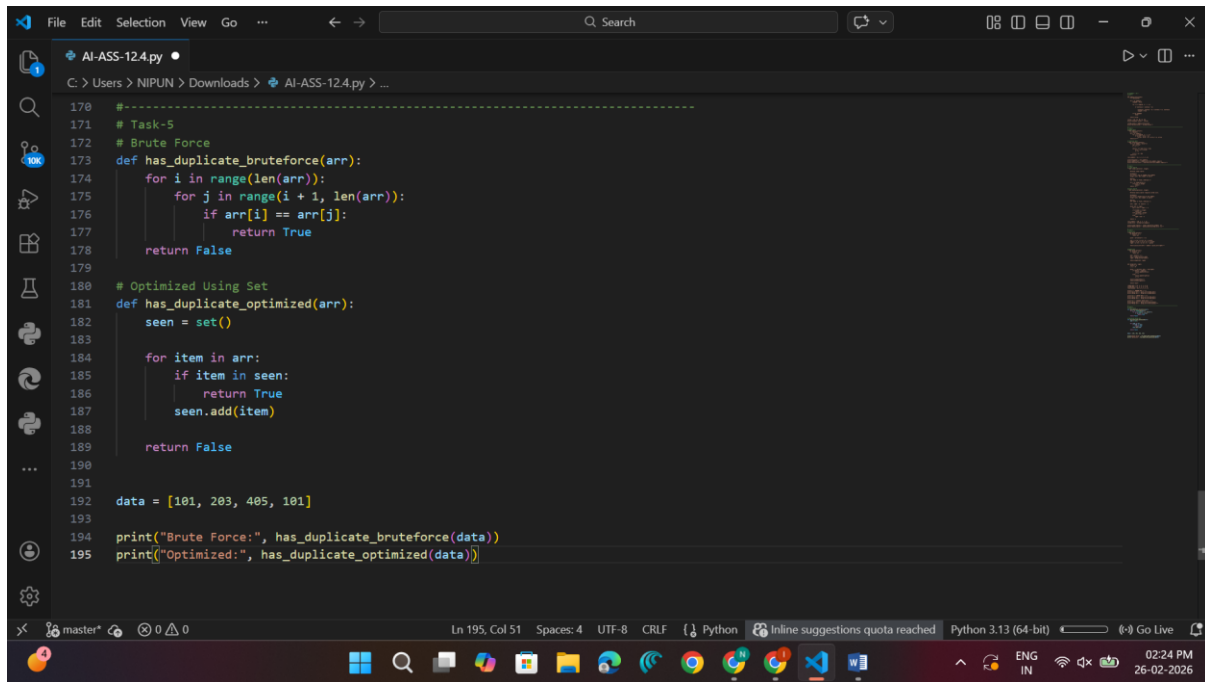
=====

TASK-5

Prompt

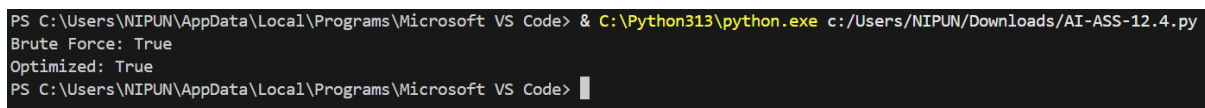
Write a python program using this three types 1)Naive duplicate detection using nested loops, 2)Use AI to optimize using sets, 3)Compare time complexity.

Code:



```
170 #-----
171 # Task-5
172 # Brute Force
173 def has_duplicate_bruteforce(arr):
174     for i in range(len(arr)):
175         for j in range(i + 1, len(arr)):
176             if arr[i] == arr[j]:
177                 return True
178     return False
179
180 # Optimized Using Set
181 def has_duplicate_optimized(arr):
182     seen = set()
183
184     for item in arr:
185         if item in seen:
186             return True
187         seen.add(item)
188
189     return False
190
191
192 data = [101, 203, 405, 101]
193
194 print("Brute Force:", has_duplicate_bruteforce(data))
195 print("Optimized:", has_duplicate_optimized(data))
```

Output:



```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-12.4.py
Brute Force: True
Optimized: True
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> |
```

Observation

- Brute Force $\rightarrow O(n^2)$
- Optimized (Set) $\rightarrow O(n)$

Set lookup takes constant time $O(1)$, reducing nested comparisons.

For large datasets, optimized method is significantly faster.